

DataFlex SSO Library v2.0.0

DATA ACCESS WORLDWIDE

Table of Contents

DataFlex SSO Library v2.0.0	1
DataFlex SSO Library v1.0.0	1
Chapter 1 Introduction	3
Chapter 2 Adding the library to your workspace	4
Chapter 3 Configuration	5
cSSOApi.....	5
cSsoProviderOpenId	5
SSO\CallbackHandler.wo	7
cSsoOnLoad_mixin.....	7
cSsoState_mixin.....	7
Chapter 4 Using the Library	10
IdentityLogin	10
IdentityLogout	10
RevokeTokens.....	10
RefreshTokens	10
GetSsoUserInfo.....	10
And many more	10

Chapter 1 Introduction

DataFlex has its own login system, and this works great. But sometimes you want a little more functionality out of your applications. For example, you want one single provider that does your login externally. This Library can help you with that. We have built a library that mostly follows the OpenID connect specification so that your DataFlex application can implement the desired SSO provider.

It was first shown at EDUC 2020 (Reykjavik, Iceland) and since then only been distributed privately. Since then it has been revamped to become version 2.0.0 and it will release on the package manager included in DataFlex 2026 (26.0).

Chapter 2 Adding the library to your workspace

The DataFlex SSO library has been designed to be both easy to use and flexible. These principles are already apparent when you add it to your workspace. From DataFlex 2026 (26.0) and forward, libraries can be added through the package manager in the studio. This library will be available as “Single Sign On” and can be installed with a single click. For older versions the library can be added by going to **Tools -> Maintain Libraries....**

Once the library is added to the workspace you can use the SSO classes in your workspace. The library does need a little bit of configuration before it is going to work. This will be explained in the next chapter.

Note: When not added through the package manager **libcrypto-3.dll** and **libcrypto-3-x64.dll** should be added to the Programs folder of your project. We should also add the **JWT library** and it's DLLs since this is a dependency.

Chapter 3 Configuration

After installing the library into our workspace, we need to setup some objects. Start by adding the `SSO\Sso.pkg` into your `WebApp.src`:

```
Define C_DEMO_PROVIDER_ID for "ENTRA"  
Use SSO\Sso.pkg
```

I would highly recommend that you also made a constant like the above code. We are going to use it multiple times. That is going to be our provider ID.

Once that is done, we can make the important objects.

cSSOApi

The `cSSOApi` is the most important class of them all. This class contains all the logic for authentication, logout and JWT functions. This Api is for that reason also available in the global handle `ghoSSO`. The object definition is not so hard with this class. We first need to make an object as follows:

```
Object oSso is a cSsoApi  
    Set psHomeURL to  
    "http://localhost/ModernSecurity"  
    Set psCallbackURLLogin to  
    "http://localhost/ModernSecurity/SSOCallback/Login"  
    Set psCallbackURLLogout to  
    "http://localhost/ModernSecurity/SSOCallback/Logout"  
End_Object
```

This object needs three properties:

- `psHomeURL` -> Needs to be the URL of the DataFlex WebApp
- `psCallbackURLLogin` -> Needs to be the URL of the DataFlex WebApp + `"/SSOCallback/Login"`
- `psCallbackURLLogout` -> Needs to be the URL of the DataFlex WebApp + `"/SSOCallback/Logout"` (Change the `CallbackHandler` code to alter this.)

These properties must be set so that the object can work properly. They will be used to find its way back to the application and to handle the request coming back from the SSO provider.

cSsoProviderOpenId

The `cSsoProviderOpenId` class is representing the SSO provider. This class is an extension of the `cSsoProvider` class and has a lot of extra properties. These properties are all used to obtain authentication from the running SSO provider. First, we need to make the object as follows:

```
Object oProviderDemo is a cSsoProviderOpenId  
End_Object
```

Then we have a whole list of properties that we can or must set:

Property	Required/Optional	Explanation
psProviderID	Required	The provider ID of your choice. You can use the constant defined earlier. This is used by the CallbackHandler to determine which provider object to return to.
psSSOHost	Required	The base URL of the SSO/identity provider, including the scheme such as https://, but without any path.
psIdentityClientId	Required	The client ID that identifies your application to the identity provider.
psIdentityClientSecret	Optional	The client secret issued by the identity provider. This is only required for confidential clients. Windows/FlexTron clients are public clients, since they cannot safely store a secret without exposing it.
psScopeParam	Optional	The scopes to request during the authorization request. If this property is omitted or left empty, behavior depends on the provider. Some providers apply a default scope, while others may reject the request or return a token with limited permissions. Refer to the provider's documentation for details.
pbAutoDiscovery	Optional	Optional, but recommended to set to true for convenience. This only works if the identity provider exposes an OpenID Connect discovery document (openid-configuration). This will allow for automatic configuration of the endpoint properties.
pbAutoLoadCertificates	Optional	Optional, but recommended to set to true for convenience. This only works if the identity provider exposes both an OpenID Connect discovery document and a JWKS endpoint. It automatically loads the provider's public signing keys into the internal JWT object.

If **pbAutoDiscovery** is not used, the following endpoint properties must be configured manually, where each endpoint may be specified either as an absolute URL or as a relative path resolved against **psSSOHost**:

Property	Explanation
psLoginEndpoint	The authorization/login endpoint used to start the authorization request and redirect the user to the identity provider's sign-in and consent page.
psTokenEndpoint	The token endpoint used to exchange the authorization code for tokens such as the access token, ID token, and optionally a refresh token.
psIntrospectEndpoint	The token introspection endpoint used to validate a token and retrieve metadata about it, if supported by the identity provider.
psLogoutEndpoint	The logout endpoint used to sign the user out at the identity provider or trigger single logout, if supported.

psUserInfoEndpoint	The user info endpoint used to retrieve user profile claims for the authenticated user, typically when using OpenID Connect.
psTokenRevocationEndpoint	The token revocation endpoint used to invalidate an access token or refresh token, if supported by the identity provider.
psJWKEndpoint	The JWKS endpoint used to retrieve the provider's public signing keys so JWTs such as ID tokens can be verified.

SSO\CallbackHandler.wo

Because we make use of the Authorization code flow, we must do some HTTP calls after someone authenticates. This is done in an Callbackhandler, this handler makes the Token request and does a user login of the session manager below. To use this Callbackhandler we must include it inside our WebApp object. Here is an example:

```
Use Dashboard.wo
Set phoDefaultView to oDashboard
Use SSO\CallbackHandler.wo
```

cSsoOnLoad_mixin

cSsoOnLoad_mixin is a small mixin class that adds SSO error handling to an object's load lifecycle.

Its SsoOnLoad procedure checks the current URL for the C_SSO_LOGIN_ERROR_PARAMETER query parameter, which is expected to contain an error returned from the login flow. If such an error is present, it shows the message to the user through UserError with the title "Login failed".

After displaying the error, it cleans up the browser URL by removing the query parameters that were added during the SSO redirect. It does this by retrieving the application's home URL from ghoSso, appending the current state hash from ghoWebApp, and then calling HistoryReplaceState so the visible URL is updated without reloading the page.

In practice, this mixin is used to detect failed SSO login attempts when a page loads, inform the user, and restore a clean application URL afterward. It should be used like this:

```
Use SSO\cSsoOnLoad_mixin
Import_Class_Protocol cSsoOnLoad_mixin

Procedure OnLoad
    Forward Send OnLoad
    Send SsoOnLoad
End_Procedure
```

cSsoState_mixin

cSsoState_mixin is a higher-level SSO state mixin built on top of cSsoStateBase_mixin. Where cSsoStateBase_mixin provides the underlying state infrastructure and lifecycle hooks, cSsoState_mixin

adds the concrete logic for storing, accessing, validating, refreshing, and clearing the active SSO identity.

The mixin introduces the private **ptIdentity** property, which stores the current **tSsoIdentity** structure. This contains the active provider ID together with the tokens and related identity data for the current session. In Windows applications this state remains synchronized naturally because everything runs in a single process, while in Web applications it is intended to be serialized as part of the broader SSO state handling.

cSsoState_mixin exposes helper functions to work with the active identity. It can return the current provider ID, report whether an identity is active, and retrieve the ID token, access token, or refresh token for a given provider. This gives the application a simple way to access the token set associated with the current login.

It also provides the operational methods needed to manage that identity. **IdentityLogout** logs the user out through the SSO API, optionally preserving the current web state hash so the user can be returned to the same view afterward. Regardless of the provider-specific logout call, it then clears the stored identity, persists the updated state, and triggers the logout success hook.

For token lifecycle management, the mixin can validate the current access token and refresh it when necessary. Validation is handled in a provider-aware way: if an expiry timestamp is available, that timestamp is used directly; otherwise the token is validated through the API. This is important for providers such as *Microsoft Entra ID*, where access tokens may be opaque and cannot always be validated locally. When a refresh succeeds, the mixin updates the stored identity with the new tokens and saves the updated state.

Finally, **cSsoState_mixin** can retrieve user information for the current identity through the configured SSO API and includes a private helper to fully clear the stored identity.

In practice, **cSsoState_mixin** serves as the main application-facing layer for SSO session state: it builds on **cSsoStateBase_mixin** and adds the logic needed to manage the authenticated user and their tokens throughout the session.

This mixin can be used in any **cObject** to make that object identity-aware. Once mixed in, the object gains access to the active SSO identity and the supporting logic needed to work with it, such as reading the current provider, retrieving tokens, validating the session, refreshing tokens, obtaining user information, and clearing the identity during logout.

A particularly useful place to apply it is a session manager. In that role, the session manager becomes the central point for tracking the authenticated SSO identity for the current user and for maintaining that identity throughout the lifetime of the session. This keeps authentication state in one well-defined place and makes it easier for the rest of the application to query login status or request token-related operations without duplicating SSO logic.

Because the mixin is built for reuse, it is not limited to session management. It can be added to any object that needs awareness of the current authenticated identity or needs to participate in SSO state handling. This makes it a flexible building block for integrating SSO into an application in a structured way.

For a practical example of how to use it, refer to the demo code. The demo shows how the mixin can be incorporated into a web application to make the application SSO-enabled, and it provides a useful reference for wiring the identity state, login flow, and session handling together.

Chapter 4 Using the Library

Now that you have configured the library correctly, u can use the login methods on any state object. Here is a better understanding of which functions there are to use.

IdentityLogin

The IdentityLogin function is used to send, or redirect, the login request to the SSO provider. This starts the full process of login and authorization. Usually, u would use this under a button by the login or at load in the webapp.

IdentityLogout

IdentityLogout is the opposite of IdentityLogin. The function first revokes your tokens and then log you out by redirecting to the SSO logout endpoint.

RevokeTokens

RevokeTokens is what it says. It revokes your tokens.

RefreshTokens

This function refreshes your tokens with the refresh token inside of **ptidentity**. It automatically replaces all the tokens for you.

GetSsoUserInfo

With the access token, this function allows us to gather information about the logged in user.

And many more

The functions above are not the only functions that we can use. There are more functions like **GetIdToken**, **GetAccessTokens** etc. Check all other functions out by going to the files in the library. Most of these functions are self-explanatory.

In the latest version we have added a few events:

- **SsoLoginSuccess**: which receives the **tSsoTokenResponse** struct which at the end should always be returned so the callback handler can handle the responses accordingly.
 - o During the SsoLoginSuccess we might need to map our external identifier to a local WebAppUser and this could go wrong, that's when we should set **tResponse.bSuccess** to false and set an error message at **tResponse.sError**.
- **SsoLoginFailed**: which receives the **tSsoTokenResponse** struct. We can simply set our custom error (**tResponse.sError**) and should move False to **tResponse.bSuccess** before returning.
- **OnSsoLogoutSuccess**: an event that gets fired after logging out the SSO session through the state object. Can be used to have any custom logic, is empty by default.
- **OnSsoLoadState**: Called to save the current SSO state. Since the callbacks come in through a **cWebHttpHandler** the library wasn't able to utilize WebProperties. That's why SsoLoginSuccess should call **OnSsoLoadState** to persist the tokens to the current session.

- **OnSsoLoadState**: Should be called for each request to load the right data into the state object properties. For example in cSSOWebSessionManager it gets called in **OnSessionPropertiesSet**. The **OnSessionPropertiesSet** event gets called during each request making this a good hook to call the **OnSsoLoadState**.